☕ What is Java?

✅ Definition

Java is a **high-level, object-oriented, platform-independent programming language** used to build:

- Web applications
- Mobile apps (Android)
- Desktop software
- Enterprise systems
- Backend APIs

🔥 Why Java is Popular?

1️⃣ Platform Independent

Java follows:

> **Write Once, Run Anywhere (WORA)**

Code is compiled into **bytecode**, which runs on the **JVM (Java Virtual Machine)**.
So the same program works on Windows, Linux, and Mac.

2️⃣ Object-Oriented

Java is based on OOP concepts:

- Class
- Object
- Inheritance
- Polymorphism
- Encapsulation

- Abstraction

3️⃣ Secure & Robust

- Strong memory management
- Exception handling
- No direct pointer access

🏗️ How Java Works?

1. You write code → `Hello.java`
2. Compiler converts it → Bytecode (`.class`)
3. JVM runs the bytecode

Flow:

Java Code → Compiler → Bytecode → JVM → Output

💻 Simple Example

```
public class Hello {
  public static void main(String[] args) {
      System.out.println("Hello Gaurav!");
  }
}
```

Output:

Hello Gaurav!

🚀 Where Java is Used?

- Android apps

- Banking systems
- Enterprise software
- Backend development (Spring Boot)
- Big systems like Amazon & Netflix

🎯 Simple Definition (Interview Ready)

> Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now owned by Oracle) that follows the principle of Write Once, Run Anywhere.

☕ Java Interview Questions (Topic: Introduction to Java)

🔹 1. What is Java?

Answer:
Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now owned by Oracle). It follows the principle of **Write Once, Run Anywhere (WORA)**.

🔹 2. Who developed Java?

Answer:
Java was developed by **James Gosling** at **Sun Microsystems** in 1995. Later, Sun Microsystems was acquired by **Oracle Corporation**.

🔹 3. Why is Java platform-independent?

Answer:
Java is platform-independent because:

- It compiles code into **bytecode**
- Bytecode runs on the **JVM (Java Virtual Machine)**
- JVM is available for multiple operating systems

So the same Java program runs on Windows, Linux, and Mac.

🔹 4. What is JVM?

Answer:
JVM (Java Virtual Machine) is a virtual machine that:

- Executes Java bytecode
- Provides platform independence
- Manages memory (Garbage Collection)

🔹 5. What is JDK?

Answer:
JDK (Java Development Kit) is a software package used to develop Java applications.
It contains:

- JRE
- Compiler (javac)
- Development tools

🔹 6. What is JRE?

Answer:
JRE (Java Runtime Environment) provides the environment to run Java programs.
It contains:

- JVM
- Core libraries

🔹 7. Difference Between JDK, JRE, and JVM?

Feature	JVM	JRE	JDK
Purpose	Runs bytecode	Runs Java programs	Develop Java programs
Contains	Execution engine	JVM + Libraries	JRE + Compiler

🔹 8. What are the main features of Java?

Answer:

- Platform Independent
- Object-Oriented
- Secure
- Robust
- Multithreaded
- Portable
- High Performance

🔹 9. What is Bytecode?

Answer:
Bytecode is the intermediate code generated after compiling Java code.
It is executed by JVM.

🔹 10. What is “Write Once, Run Anywhere”?

Answer:
It means Java code can be written once and executed on any system that has a JVM.

🔹 11. Is Java purely object-oriented?

Answer:
No. Java is not purely object-oriented because it supports primitive data types like int, float, char, etc.

🔹 12. Why is Java secure?

Answer:

- No pointer access
- Bytecode verification
- Security manager
- Automatic memory management

🔹 13. What is the difference between C++ and Java?

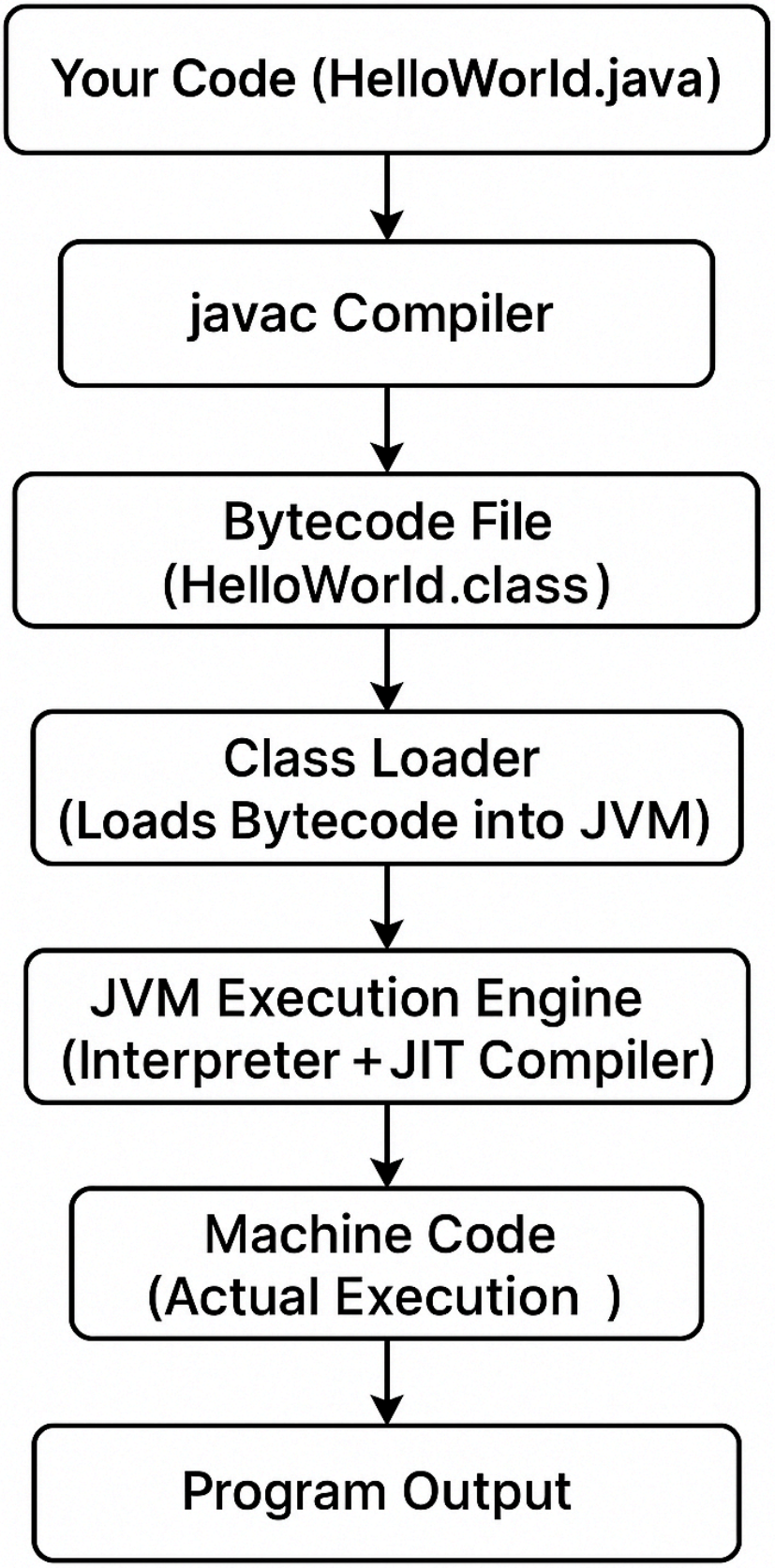
C++	Java
Platform dependent	Platform independent
Supports pointers	No direct pointer support
Multiple inheritance	Uses interfaces

🎯 Important Interview Tip

If interviewer asks:
👉 "Explain Java in 1 minute"

Say:

> Java is a high-level, object-oriented, platform-independent programming language developed by James Gosling in 1995. It follows the principle of Write Once, Run Anywhere using JVM. It is widely used for web, enterprise, and Android development.


Your Code (HelloWorld.java)
javac Compiler
Bytecode File
(HelloWorld.class)
Class Loader
(Loads Bytecode into JVM)
JVM Execution Engine
(Interpreter + JIT Compiler)
Machine Code
(Actual Execution)
Program Output